PROGRAMMING FOR AI · FINAL PROJECT · TOPIC 37

Forest Cover Type Classification

Predicting the dominant tree species of a 30 × 30 m cell from terrain and soil alone

Amanbaev Marlen AAI-2601
Erik [surname] [group]
Rashad [surname] [group]

November 2026

Agenda

01 **Problem & data**
581k cells, 7 classes, strong imbalance

02 **Data engineering**
chunked loading, dtypes, Parquet, audit

03 **Analysis & SQL**
EDA, normalised SQLite, 12 queries

04 **Modelling**
leakage-free protocol, 5 approaches

05 **Results**
quality vs cost, errors, demo, conclusions

THE QUESTION

Which approach classifies forest cover best, and at what cost in time and memory?

Baseline · Logistic Regression · Random Forest · Gradient Boosting, compared by F1-macro, confusion matrix, time, RAM and model size

581,012 cells, and one class is 103× rarer than another

581k

30 × 30 m cells (UCI Covertype)

54

features: 10 numeric, 4 areas, 40 soils

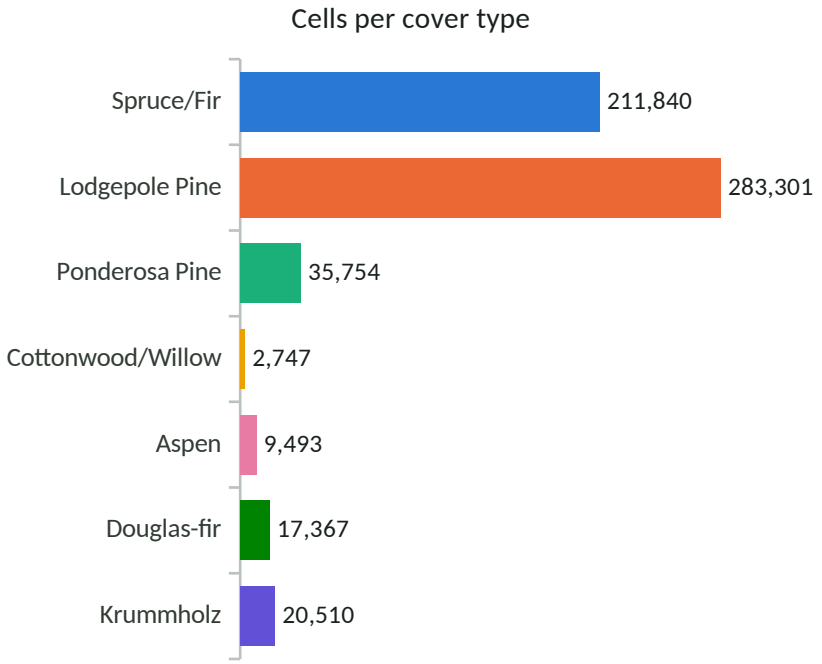
7

forest cover types to predict

103×

Lodgepole Pine vs Cottonwood/Willow

So the main metric is F1-macro: every class counts the same. Always predicting the majority class already gives 49% accuracy, but F1-macro 0.09.



One reproducible pipeline in twelve modules

1. **Load** — `data_loader.py` — chunks, dtypes, Parquet
2. **Audit** — `audit.py` — quality checks
3. **Explore** — `eda.py` — figures, Plotly
4. **Store** — `database.py` — SQLite, 12 queries
5. **Model** — `models.py` — pipelines, CV tuning
6. **Explain** — `evaluate.py` — importance, errors

Two interfaces

main.py console menu with 9 operations

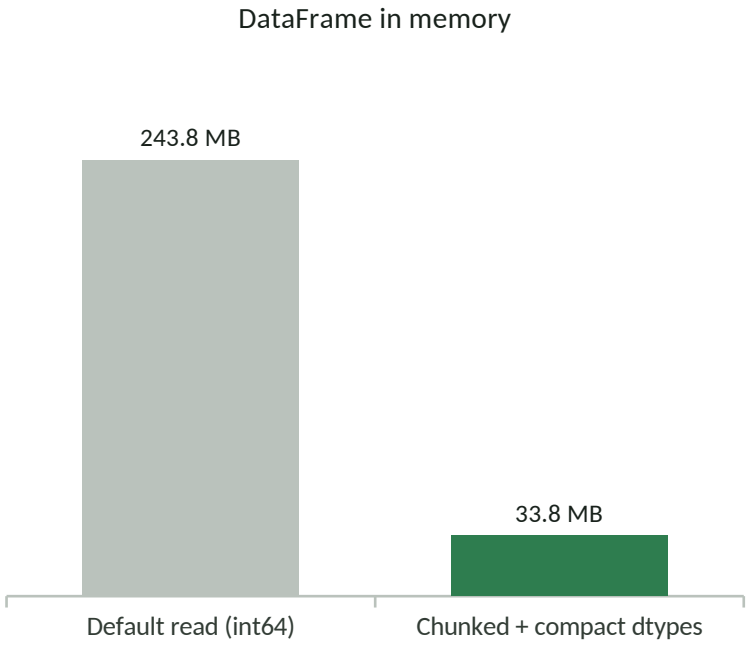
app.py Streamlit dashboard: filters, SQL, models, prediction form

One command

python main.py --all reproduces every number in this deck

config.py all paths and seeds; 6 pytest tests

Compact dtypes cut memory by 86%; Parquet is 10× smaller



Format	File	Load	Peak RAM
CSV, default	71.69 MB	0.803 s	487.8 MB
CSV, compact dtypes	71.69 MB	0.691 s	127.7 MB
Parquet	**6.91 MB**	**0.152 s**	**7.3 MB**
Parquet, 3 columns	6.91 MB	0.007 s	1.1 MB

Chunks of 100,000 rows with int16 / uint8: no value is lost

44 one-hot columns collapsed into 2 categorical columns

Columnar Parquet reads 3 columns in 7 ms

The audit found clean data, so no rows were dropped

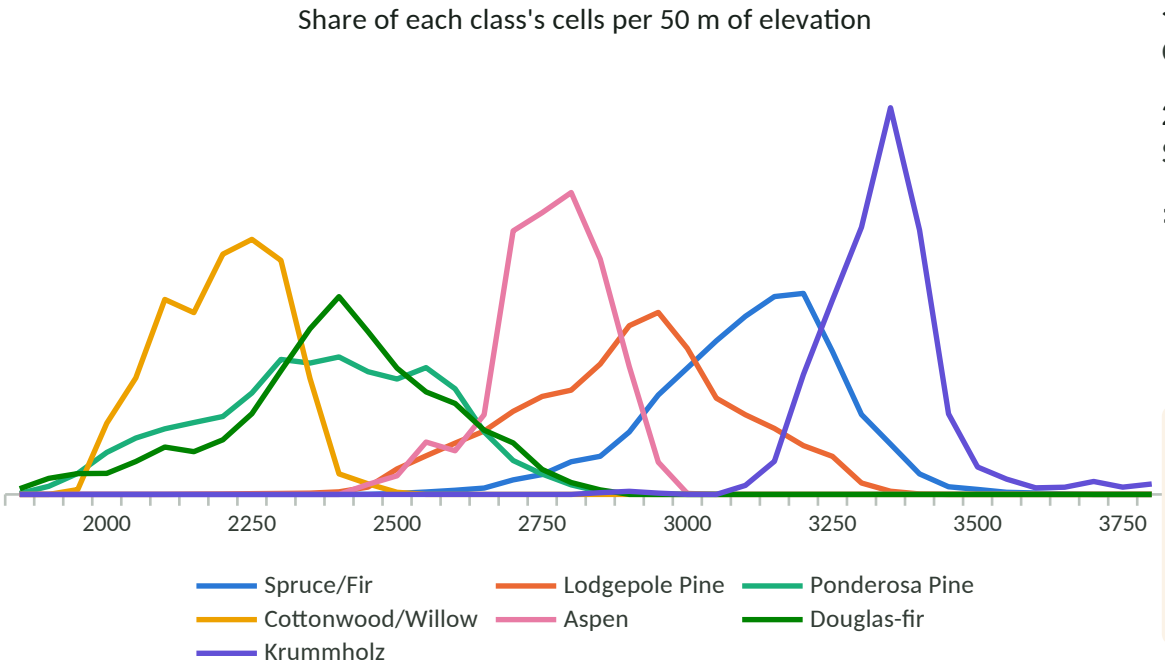
✓	Missing values	**0**	imputers only guard user input
✓	Duplicates / conflicting labels	**0 / 0**	no label noise
✓	Rows with 0 or > 1 soil flag	**0**	one-hot safe to collapse
✓	Values outside physical range	**0**	nothing to fix
!	IQR outliers, worst column	**5.42%**	kept: real terrain extremes
!	Hillshade 3 pm = 0	**1,338**	kept: afternoon shadow

DECISION

Keep every row.

Outliers are remote cells and steep slopes. Removing them would delete the rare high-altitude classes.

Each species occupies its own altitude belt



< 2,600 m Ponderosa, Douglas-fir, Cottonwood/Willow

2,600–3,300 m Lodgepole Pine, Aspen, Spruce/Fir

> 3,300 m Krummholz at the tree line

The belts overlap: elevation alone cannot separate Spruce/Fir from Lodgepole Pine.

Wilderness area and soil act as strong priors

% of each area's cells by cover type



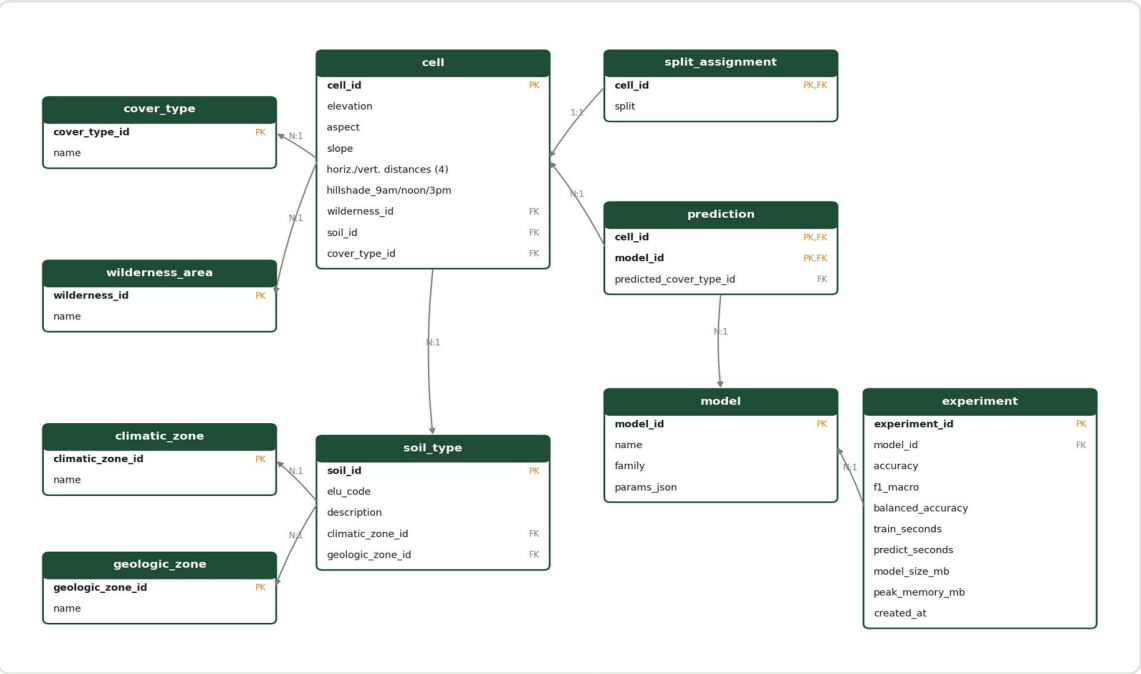
Cottonwood/Willow exists only in Cache la Poudre; Neota is 62% Spruce/Fir.

91%

of soil 12 (29,971 cells) is Lodgepole Pine · SQL Q03

Classes are defined by combinations of elevation × area × soil. That is what tree models capture and a linear model cannot.

SQLite keeps data, models and predictions together



10 tables 44 one-hot columns → 2 foreign keys

12 queries JOINs over up to 4 tables, GROUP BY / HAVING

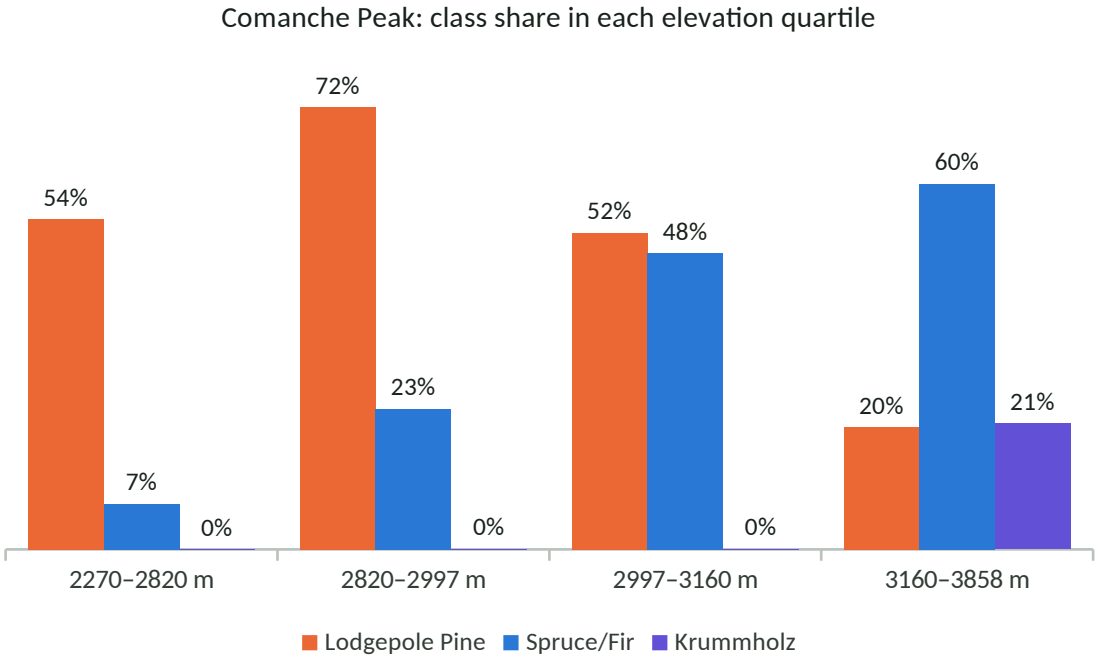
CTE & subqueries in 8 queries

Window functions SUM OVER, ROW_NUMBER, RANK, NTILE, LAG

Errors in SQL test predictions stored next to the data

pandas and SQL give identical aggregates (17 groups)

SQL window functions reveal the altitude succession




```
NTILE(4) OVER (
  PARTITION BY wilderness_id
  ORDER BY elevation)
```

Low quartiles Lodgepole Pine dominates

Upper quartiles Spruce/Fir takes over

Top quartile Krummholz appears (21%)

A leakage-free protocol, from split to single test

01

Split first

stratified 80 / 20
464,809 / 116,203

02

Pipelines

features, imputing,
scaling fit on train

03

Tune

3-fold CV, F1-macro
40k-row subsample

04

Refit

best params on the
full train split

05

Test once

quality, time,
RAM, size

BASELINE

Dummy: majority class

LINEAR

Logistic Regression

BAGGING

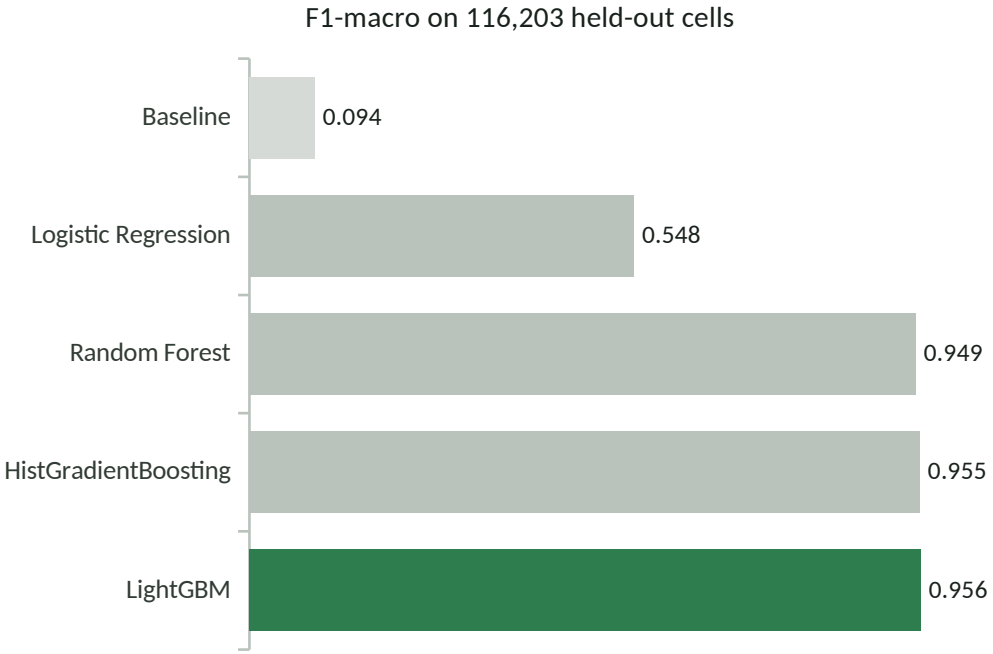
Random Forest

BOOSTING

HistGB, LightGBM

Imbalance: class_weight ∈ {None, balanced} is searched, plus a controlled LightGBM check with the opposite weighting.

Gradient boosting wins with F1-macro 0.956



0.974

accuracy of LightGBM (baseline 0.488)

+0.41

F1-macro of trees over the linear model

Trees learn interactions of elevation, area and soil; a linear boundary cannot.

Quality is close at the top, so cost decides

Model	F1-macro	Accuracy	Train	Predict / 1k rows	Size
Logistic Regression	0.548	0.731	43 s	1 ms	0.01 MB
Random Forest	0.949	0.972	62 s	10 ms	568 MB
HistGradientBoosting	0.955	0.970	328 s	101 ms	60 MB
LightGBM	**0.956**	**0.974**	**172 s**	**402 ms**	**91 MB**

LightGBM

best F1-macro, 91 MB in memory, 40 MB on disk: selected

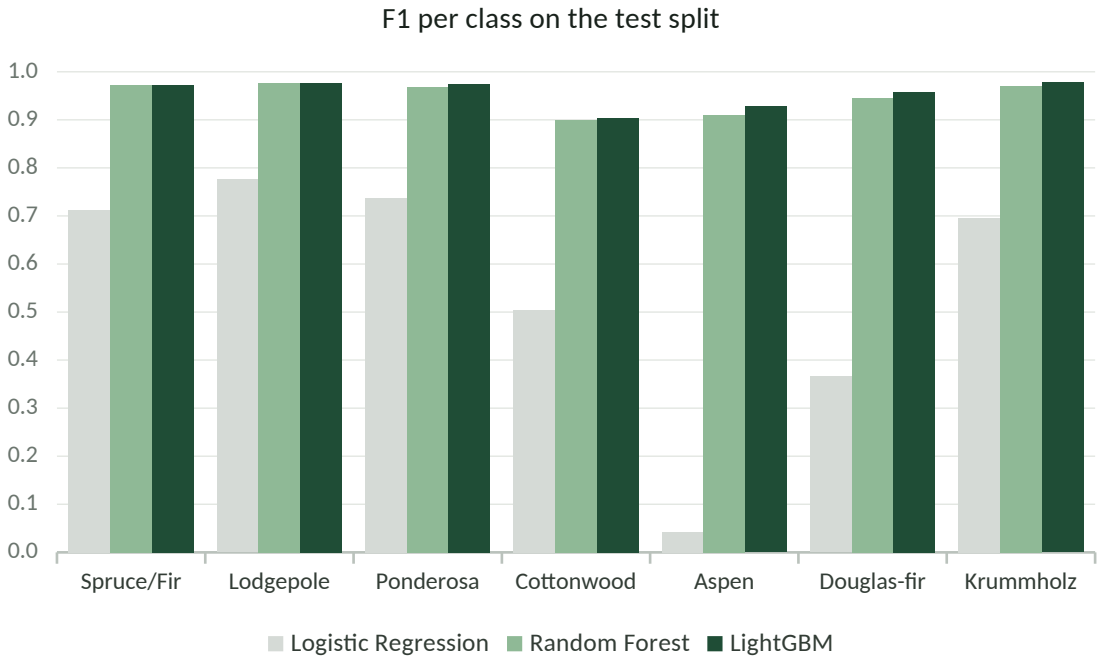
Random Forest

3× faster to train, 39× faster to predict, but 568 MB

HistGB

close second, smallest tree model, 2× slower to train

Rare classes separate the models



CLASS WEIGHTS

Balanced acc. **0.947 → 0.959**
Cottonwood F1 **0.889 → 0.904**
Accuracy **unchanged, 0.974**

LESSON FROM TUNING

HistGB with lr 0.2 + balanced weights scored 0.8421 in CV, then diverged on all data (0.3827). Capping lr at 0.1 fixed it (0.955).

Errors fall between neighbouring vegetation belts

	Spruce/Fir	Lodgepole	Ponderosa	Cottonwood	Aspen	Douglas-fir	Krummholz
Spruce/Fir	**97.4%**	2.3%					0.2%
Lodgepole	2.0%	**97.4%**	0.1%		0.3%	0.1%	
Ponderosa		0.7%	**97.8%**	0.4%	0.1%	1.0%	
Cottonwood			6.0%	**89.6%**		4.4%	
Aspen	0.6%	3.7%	0.5%		**94.8%**	0.4%	
Douglas-fir		0.7%	2.6%	0.6%	0.1%	**96.1%**	
Krummholz	1.5%	0.3%					**98.2%**

Rows: true class · columns: predicted · cells: % of the true class

69%

of 3,063 errors: Spruce/Fir ↔ Lodgepole

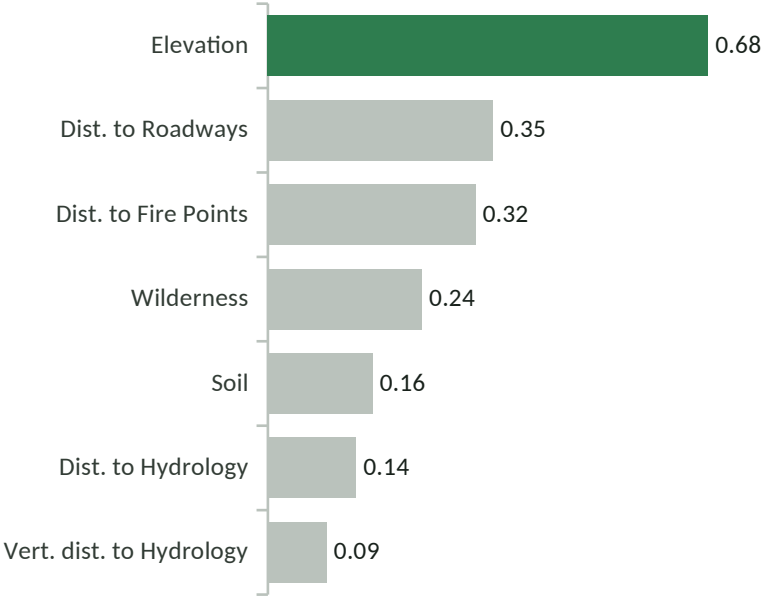
45%

error rate when confidence < 0.6 (vs 1.9%)

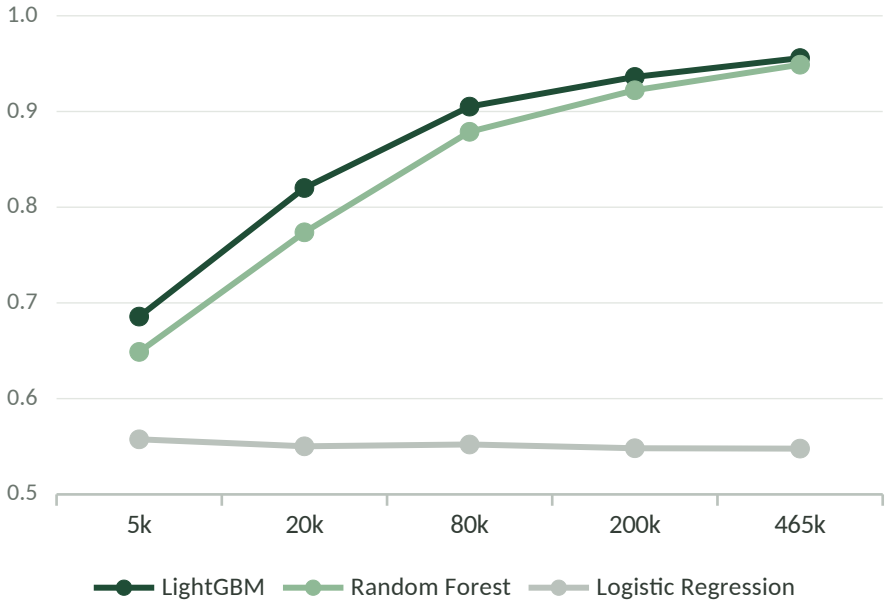
Low-confidence cells (1.7%) are natural candidates for a field check.

Elevation matters most, and more data keeps helping

Permutation importance (drop in F1-macro)



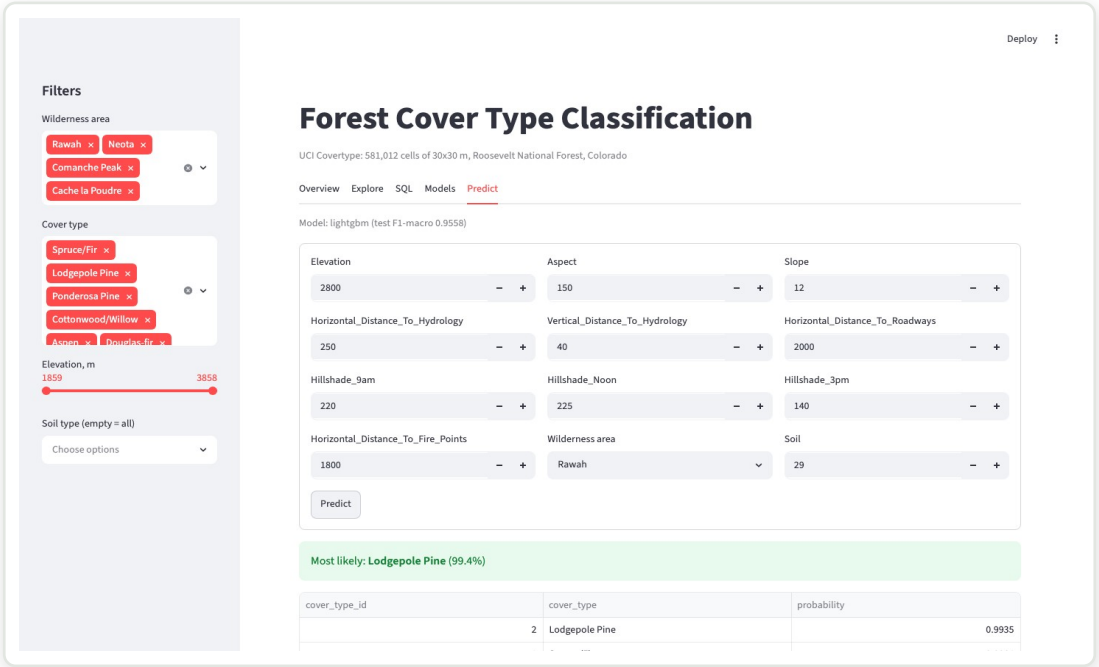
F1-macro vs training rows



Live demo: console menu and Streamlit dashboard

```
$ python main.py

1 Prepare data
2 Data quality audit
3 Build SQLite DB
4 EDA figures
5 Train & compare
6 Interpret model
7 Run SQL query
8 Predict
9 Results
0 Exit
```



CONCLUSIONS

Terrain and soil are enough to map the forest

0.956

F1-macro of LightGBM with class weights, accuracy 0.974

+0.41

F1-macro of tree ensembles over the linear model: classes are interactions

69%

of errors are between two species of the same altitude belt

Limitation: the random split ignores spatial autocorrelation between neighbouring cells, so scores are optimistic for a completely new region.

Team and contributions

AM

Amanbaev Marlen

AAI-2601

Data engineering & application

Chunked loading, dtypes, Parquet; SQLite schema, ER diagram, 12 SQL queries; console menu, Streamlit, README, tests

E

Erik [surname]

[group]

Audit & exploratory analysis

Quality checks and cleaning decisions; all EDA figures and the interactive Plotly chart; report sections 2–5

R

Rashad [surname]

[group]

Modelling & interpretation

Feature pipelines; CV tuning of 5 approaches; importance, learning curve, error analysis; report sections 6–9

Report and slides were written together; every member can explain every module.